

سیستم ذخیره سازی دیتابیس

مقدمه :

سیستم‌های شبکه اجتماعی و پیام‌رسان در مقیاس بزرگ نیازمند پردازش بلادرنگ روابط میان کاربران، گروه‌ها، کانال‌ها و محتوا هستند. پایگاه‌داده‌های سنتی رابطه‌ای یا حتی برخی NoSQL‌ها در پاسخ‌گویی کم‌تأخیر به کوئری‌های گرافی مانند بررسی ارتباط، همسایگی، دنبال‌کننده/دنبال‌شوندگی، دوستان مشترک و پیمایش‌های محدود، با محدودیت مواجه‌اند.

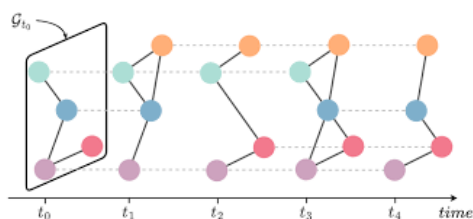
از سوی دیگر، اجرای مستقیم تمام تحلیل‌ها روی یک گراف زنده در RAM نیز مشکلاتی مانند مصرف بالای حافظه، پیچیدگی همزمانی و اختلال در سرویس‌دهی بلادرنگ ایجاد می‌کند.

بنابراین مسئله اصلی این پروژه، طراحی یک معماری گرافی هیبریدی است که:

- هم کوئری‌های بلادرنگ را با latency پایین پاسخ دهد،
- هم پایداری داده و بازیابی سریع را تضمین کند،
- و هم امکان اجرای پردازش‌های سنگین را بدون اختلال در مسیر سرویس‌دهی فراهم سازد.

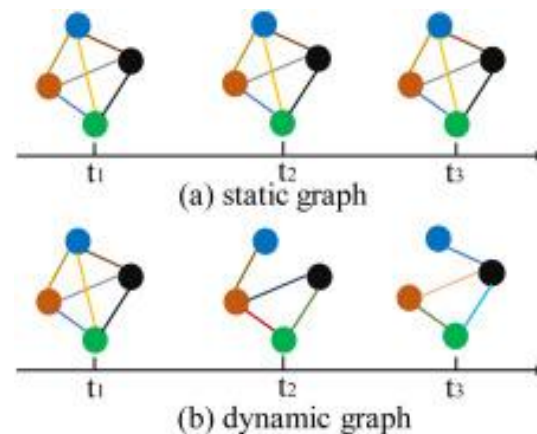
گراف پویا/داینامیک (live) :

گراف پویا (Dynamic/Live Graph) در RAM به ساختاری از داده‌های گرافی گفته می‌شود که نودها و یال‌های آن در حافظه RAM نگهداری شده و امکان اعمال تغییرات نزدیک به لحظه، مانند افزودن، حذف یا به‌روزرسانی روابط، روی آن وجود دارد. این رویکرد باعث کاهش شدید latency در query و traversal می‌شود، اما در مقابل چالش‌هایی مانند مدیریت همزمانی، consistency، مصرف حافظه و دشواری اجرای الگوریتم‌های global روی گراف در حال تغییر را به همراه دارد.



گراف Static :

گراف Static به گرافی گفته می‌شود که ساختار آن شامل نودها، یال‌ها و ویژگی‌های اصلی، در طول اجرای پردازش یا الگوریتم ثابت باقی می‌ماند. این مدل برای تحلیل‌های گرافی و اجرای الگوریتم‌های batch بسیار مناسب است، زیرا امکان استفاده از ساختارهای فشرده، پردازش سریع‌تر و نتایج پایدارتر را فراهم می‌کند.

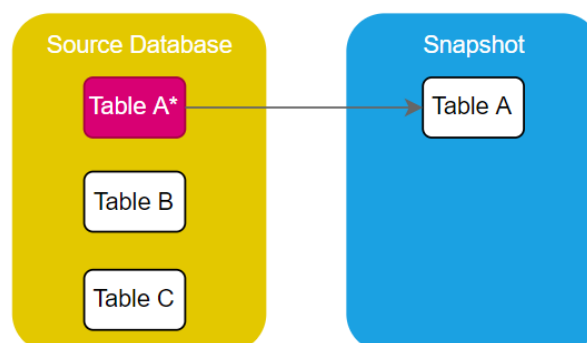


Snapshot :

Snapshot به یک نمای ثابت و سازگار از داده‌ها در یک لحظه مشخص گفته می‌شود.

در سیستم‌هایی که داده‌ها به صورت مداوم تغییر می‌کنند، Snapshot این امکان را می‌دهد که عملیات خواندن، تحلیل یا پشتیبان‌گیری روی یک نسخه پایدار انجام شود، بدون اینکه تغییرات همزمان باعث ناهمبستگی در نتیجه شوند.

به عبارت دیگر، Snapshot یک «عکس لحظه‌ای» از وضعیت داده‌ها است که برای پردازش‌های تحلیلی، بازیابی و کنترل سازگاری بسیار کاربرد دارد.



: Rocks DB

Rocks DB یک پایگاه داده کلید-مقدار (Key-Value) توکار و بسیار پرسرعت است که برای ذخیره سازی داده در حجم بالا و با نرخ زیاد نوشتن طراحی شده است.

این موتور بر پایه معماری LSM-Tree کار می کند و به همین دلیل برای سناریوهایی که نیاز به:

- درج سریع داده
- خواندن مؤثر
- مصرف بهینه دیسک
- پشتیبانی از Snapshot

دارند، گزینه ای بسیار مناسب محسوب می شود.

در این پروژه، Rocks DB به عنوان لایه ذخیره سازی پایدار انتخاب شده است تا داده ها با کارایی بالا و تأخیر کم ذخیره و بازیابی شوند.

★ بخش اصلی داده در داخل دیتابیس ما درون Rocks DB ذخیره خواهد شد .

ذخیره سازی Document-Based روی Rocks DB:

اگرچه Rocks DB ذاتاً یک دیتابیس Key-Value است، اما می توان با تعریف یک لایه منطقی روی آن، داده ها را به شکل Document-Based ذخیره کرد.

در این روش، هر رکورد به صورت یک داکيومنت کامل نگهداری می شود؛ به طوری که:

- **Key** معرف شناسه یکتا یا مسیر دسترسی به داده است
 - **Value** شامل محتوای کامل سند به صورت باینری یا ساختاریافته است
- این رویکرد باعث می شود مزایای Rocks DB در سرعت و کارایی حفظ شود، در حالی که مدل داده برای توسعه دهنده شبیه به یک سیستم سندمحور (Document Store) باقی بماند.
- در نتیجه، ساختار داده ها انعطاف پذیرتر شده و مدیریت فیلدهای متغیر یا نیمه ساخت یافته ساده تر می شود.

:BSON

BSON مخفف **Binary JSON** است و یک فرمت باینری برای ذخیره و انتقال داده‌های سندی محسوب می‌شود.

BSON از نظر مفهومی شبیه JSON است، اما به جای نمایش متنی، داده‌ها را به صورت باینری ذخیره می‌کند. این ویژگی چند مزیت مهم دارد:

- سرعت بیشتر در parse و serialization
 - حجم مناسب برای ذخیره سازی
 - پشتیبانی از انواع داده بیشتر نسبت به JSON معمولی
 - مناسب برای ذخیره مستقیم داکيومنت‌ها در لایه‌هایی مانند Rocks DB
- در این پروژه، داکيومنت‌ها به صورت BSON ذخیره می‌شوند تا هم ساختار سندی داده‌ها حفظ شود و هم کارایی ذخیره سازی و بازیابی بهبود یابد.



بیان مسئله :

ذخیره سازی گراف در داخل ram از مهمترین بخش های دیتابیس هست ، قلب تپنده هر دیتابیس سیستم کنترل و مدیریت داده ها است ، سیستمی که مسئول اصلی کنترل ، حفظ پیایداری و اجرای عملیات ها رو داده ها است . با توجه به اهمیت این سیستم ما نیاز به ساختار و معماری مناسب با توجه به هدف دیتابیس خواهیم داشت این ساختار باید به نحوی باشد که سرعت عملیات های حذف ، درج ، ذخیره سازی ، جستجو ، اجرای الگوریتم ها و بهینه و مناسب باشه .

اکثر الگوریتم های گراف برای اجرا نیاز به اسکن بخش قابل توجهی از گراف دارد اگر گراف با ساختمان داده بهینه داخل ram به صورت live یا Static موجود باشد اسکن گراف اسان و سریع تر خواهد بود هزینه I/O و هزینه رندر گراف در ram کاهش خواهد یافت .

این روش اما خالی از معایب نیست ، اولین مشکل توسعه سیستم هماهنگی گراف موجود در RAM با داده های ذخیره شده در دیسک است. دومین مشکل ظرفیت ذخیره سازی محدود RAM نسبت به حافظه ثابت در دیسک هست که میتواند سایر سرویس های روی سرور را تحت تاثیر قرار دهد . سومین مشکل انتخاب ساختمان داده ایی خواهد بود که کمترین پیچیدگی زمانی رو برای اجرای عملیات های مورد نیاز ما داشته باشد . چهارمین مشکل افزودن سیستمی برای حفظ یکپارچگی داده ها در زمان Crash برنامه یا قطع برق خواهد بود . پنجمین مشکل عدم اجرای الگوریتم ها روگراف Dynamic هست زیر گراف هیچ ساختار مشخصی ندارد و ممکن هست زمان اجرای الگوریتم یال ها و گره ها تغییر کند .

به طوری کلی اکنون میتوان گفت ذخیره گراف درون حافظه RAM بدون مشکل نخواهد بود اما میتوان راهکارهایی را دنبال کرد که این مشکلات را حذف کند یا به طور قابل توجهی کاهش دهد. از جمله این راهکارها میتوان به استفاده از ساختمان داده و معماری مناسب اشاره کرد.

راهکار پیشنهادی:

۱) مشکل شماره ۱ : هماهنگی همزمان دو منبع اصلی (Source of Truth) برای یک دیتابیس امری چالش برانگیز بوده و هست از میان راه های موجودی استفاده از سیستمی برای حفظ پایداری نسبی داده به نام WAL هست که مخفف Write-Ahead Log است و به معنی ثبت تغییرات قبل از اعمال نهایی آن ها روی داده اصلی می باشد.

ایده اصلی WAL این است:

قبل از اینکه تغییرات روی ساختار اصلی دیتابیس اعمال شوند، ابتدا همان تغییرات به صورت ترتیبی داخل یک فایل لاگ پایدار روی دیسک نوشته می شوند.

به این ترتیب اگر سیستم دچار Crash، قطع برق یا توقف ناگهانی شود، دیتابیس می تواند بعد از راه اندازی مجدد، تغییرات ثبت شده در WAL را بخواند و وضعیت داده ها را بازیابی کند.

تاریخچه کوتاه WAL :

مفهوم WAL از سیستم های پایگاه داده قدیمی و تراکنشی آمده است. دیتابیس هایی مانند PostgreSQL ، MySQL/InnoDB ، RocksDB ، SQL Server از نوعی مکانیزم مشابه WAL استفاده می کنند. در گذشته، مشکل اصلی دیتابیس ها این بود که اگر هنگام نوشتن داده سیستم خاموش می شد، ممکن بود بخشی از داده روی دیسک نوشته شده باشد و بخشی دیگر نه. این وضعیت باعث خراب شدن داده یا inconsistent شدن دیتابیس می شد.

عملکرد WAL به این صورت است که اول تغییر را در یک فایل ترتیبی و ساده ثبت میکنیم، بعداً آن را روی ساختار اصلی دیتابیس اعمال میکنیم.

مشکل اصلی که سیستم WAL آن را درست میکند پایداری داده در زمان های بحرانی هست .

اگر دیتابیس بعد حادثه بحرانی مجدد اجرا شود ، سیستم لاگ های موجود در WAL را خوانده و تغییرات درست را اعمال میکند.

یکی از ویژگی های مهم دیتابیس ها در مدل ACID، ویژگی Durability است.

Durability به این معناست که وقتی دیتابیس اعلام می کند یک عملیات با موفقیت انجام شده، آن داده نباید با Crash یا قطع برق از بین برود.

WAL ابزار اصلی برای رسیدن به Durability است. بدون WAL ، اگر داده فقط در RAM باشد، با خاموش شدن سیستم از بین می رود. اما وقتی تغییر در WAL روی دیسک ثبت شود، حتی اگر RAM پاک شود، می توان داده را بعداً بازیابی کرد.

سرعت نوشتن در WAL به دلیل append-only بودن بالاست یعنی دیتابیس برای هر تغییر دنبال جای خاصی در دیسک نمی گردد ، فقط انتهای فایل لاگ را باز می کند و رکورد جدید را به آخر آن اضافه می کند. این نوع نوشتن بسیار سریع تر از نوشتن تصادفی در نقاط مختلف دیسک است. به همین دلیل WAL هم سرعت نوشتن را حفظ می کند و هم امنیت داده را بالا می برد.

مزایای WAL :

- افزایش پایداری داده ها
- امکان بازیابی بعد از Crash
- جلوگیری از خراب شدن داده در عملیات ناقص
- سرعت بالا به دلیل append-only بودن
- امکان پیاده سازی transaction

معایب و هزینه های WAL :

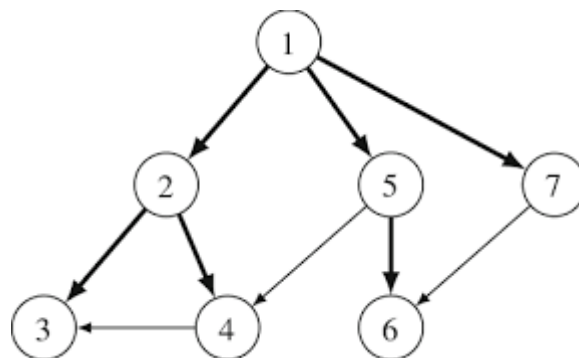
- مصرف فضای اضافه روی دیسک
- افزایش تعداد write ها
- نیاز به مدیریت و حذف WAL های قدیمی
- پیچیدگی در طراحی

۲) مشکل شماره ۲: در یک اپلیکیشن پیام رسان یا شبکه اجتماعی حجم عظیمی از داده در روز ذخیره و فراخوانی میشود، این داده ها شامل پست ها، پیام ها و است که مدیریت آنها در سمت دیتابیس همواره چالش هایی دارد.

اگر دیتابیس گرافی ما در موقع بیلد کردن یا موقع رندر کردن داده هایی اضافی تر از نیازش را وارد ساختمان داده خود کرده و در RAM ذخیره کند، گرافی به شدت سنگین و پرحجم خواهیم داشت که مدیریت آن پیچیده هست از طرفی این نوع ذخیره سازی میتواند حجم بالایی از RAM را بی دلیل اشغال کند و نیاز به سخت افزار پیشرفته برای اجرای آن را به دنبال داشته باشد.

یکی از راهکار های موجود عدم ذخیره هیچ اضافی و ذخیره داده با وسواس داخل RAM است، این جمله به این معناست که داده ما همگی در یک دیتابیس پایه ذخیره شود و فقط از مقدار یکتا داده ها گراف ساخته شود. برای نمونه برای نشان داده ارتباط کاربران با یکدیگر داده های اصلی شامل name, password, username و...

در دیتابیس اصلی ذخیره و مدیریت شود، در زمان ساخت گراف کاربر میتواند در کوئری خود id یکتا کاربر را به عنوان نماینده کاربر در هر گره گراف ذخیره کرده تا نیازی به ذخیره سایر ویژگی های کاربر در RAM نباشد ولی همچنان یکپارچگی داده ها حفظ شود. در دیتابیس ما نقش دیتابیس اصلی را Rocks DB برعهده دارد.



```
Korys-MacBook-Pro:~ korydondzila$ curl http://localhost:3000/users
[
  {
    "id": 1,
    "userId": 101,
    "name": "Alice"
  },
  {
    "id": 2,
    "userId": 102,
    "name": "Bob"
  },
  {
    "id": 3,
    "userId": 103,
    "name": "Carol"
  }
]
Korys-MacBook-Pro:~ korydondzila$
```

۳) مشکل شماره ۳: انتخاب ساختمان داده مناسب از جنبه های مختلف حائز اهمیت است. ساختمان داده مناسب میتواند اطلاعات را با حجم کمتری در حافظه قرار دهد و انجام عملیات ها و الگوریتم ها تسریع کند. نکته مهم بعدی این است ساختمان داده انتخابی ما سرعت بالایی در کپی شدن داشته باشد تا رندر شدن در RAM و گرفتن Snapshot لحظه سریع تر باشد.

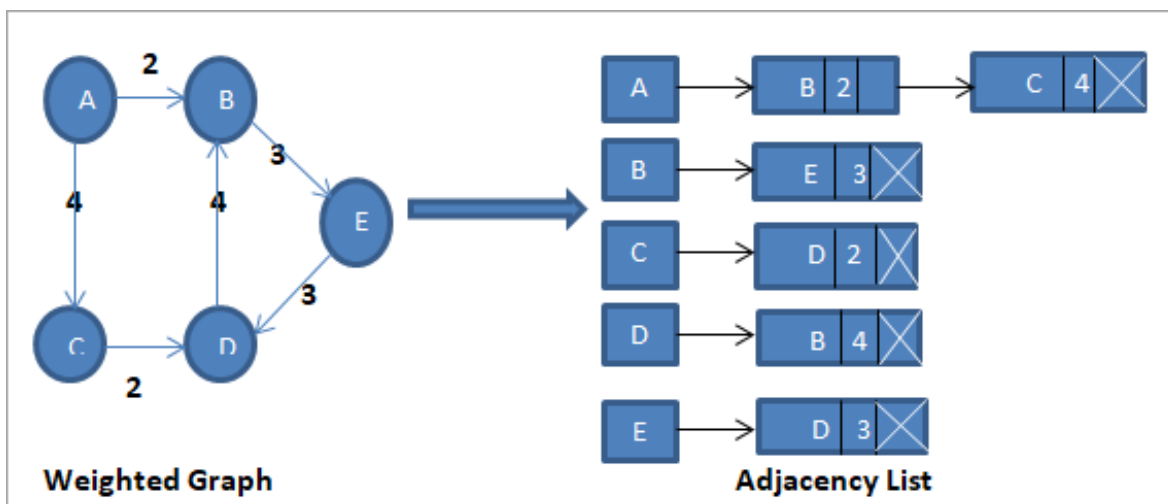
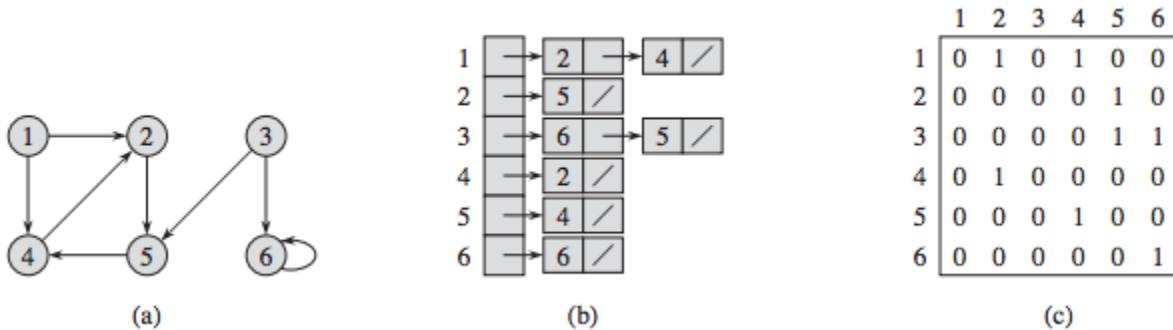
ساختمان داده Adjacency List : فهرست مجاورت یا Adjacency List یکی از ساختارهای داده ای رایج و کارآمد برای نمایش و ذخیره سازی گراف در حافظه ی اصلی (RAM) است. در این روش، برای هر رأس گراف، مجموعه یا لیستی از رأس های مجاور آن نگهداری می شود. به عبارت دیگر، به جای ذخیره سازی تمام حالت های ممکن ارتباط بین رأس ها، تنها یال های واقعی موجود در گراف ثبت می شوند.

در پیاده سازی این ساختار، معمولاً از آرایه، لیست، بردار یا جدول درهم سازی استفاده می شود؛ به طوری که هر خانه یا کلید نشان دهنده ی یک رأس بوده و مقدار متناظر با آن شامل لیستی از همسایه های آن رأس است. در گراف های وزن دار، علاوه بر شناسه ی رأس مجاور، وزن یال نیز در هر ورودی ذخیره می شود. همچنین در گراف های جهت دار، ارتباط ها بر اساس جهت یال ها ثبت می شوند، در حالی که در گراف های بدون جهت، هر یال معمولاً در لیست مجاورت هر دو رأس ذخیره می گردد. از نظر مصرف حافظه، فهرست مجاورت نسبت به روش هایی مانند ماتریس مجاورت کارایی بیشتری دارد، به ویژه زمانی که گراف پراکنده باشد. پیچیدگی حافظه این روش برابر با $O(V + E)$ است که در آن V تعداد رأس ها و E تعداد یال های گراف را نشان می دهد. این در حالی است که ماتریس مجاورت به

فضایی در حدود $O(V^2)$ نیاز دارد. بنابراین، برای گراف‌های بزرگ با تعداد یال‌های محدود، استفاده از فهرست مجاورت باعث کاهش قابل توجه مصرف حافظه در RAM می‌شود.

از نظر عملکرد، این ساختار داده برای پیمایش گراف و اجرای الگوریتم‌هایی مانند جست‌وجوی عمقی (DFS)، جست‌وجوی سطحی (BFS) و الگوریتم‌های مسیریابی مانند دایکسترا بسیار مناسب است، زیرا دسترسی به همسایه‌های هر رأس به صورت مستقیم انجام می‌شود. با این حال، بررسی وجود یال مستقیم بین دو رأس مشخص ممکن است نیازمند جست‌وجو در لیست همسایه‌های یک رأس باشد و در مقایسه با ماتریس مجاورت زمان بیشتری مصرف کند.

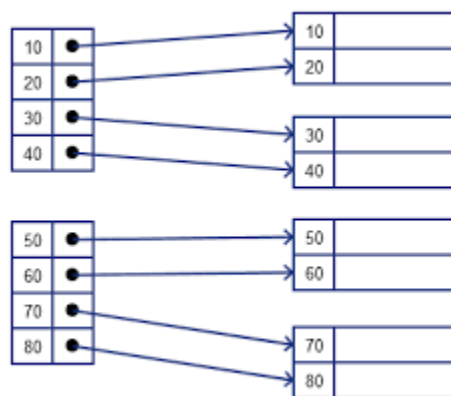
در مجموع، فهرست مجاورت به دلیل مصرف بهینه‌ی حافظه، سادگی پیاده‌سازی و سازگاری با الگوریتم‌های متداول پردازش گراف، گزینه‌ای مناسب برای ذخیره‌سازی گراف در حافظه‌ی اصلی محسوب می‌شود؛ به خصوص در شرایطی که گراف دارای تعداد زیادی رأس و تعداد نسبتاً محدودی یال باشد.



بهینه سازی های لازم برای بهینه سازی ساختمان داده adjacency list ضروری است ، زیرا همانطور که قبل تر اشاره شد عملیات های درج و حذف در این ساختمان داده بهینه نیست .

یکی از راه های بهینه سازی مرتب سازی اعضای وکتور مربوط به ارتباط گره ها است ، برای این امر نیاز به این است ، اعدادی که نشانگر گره ها هستن ساختار عددی درست داشته باشند .

در طراحی ساختار ذخیره سازی گراف در حافظه ی اصلی، ابتدا شناسه ی نودها به صورت Dense-ID بازنمایی می شود. در این روش، شناسه های اصلی نودها که ممکن است رشته ای، بزرگ یا پراکنده باشند، به شناسه های عددی کوچک و پیوسته نگاشت می شوند. این کار امکان استفاده از آرایه های ساده برای دسترسی مستقیم به اطلاعات هر نود را فراهم کرده و موجب کاهش سربار حافظه، افزایش سرعت دسترسی و بهبود locality در حافظه می شود.



برای نودهای معمولی که درجه ی آن ها کم یا متوسط است، لیست همسایگی به صورت sorted vector ذخیره می شود. در این روش، شناسه ی همسایه ها در یک آرایه ی پویا و مرتب قرار می گیرد. مرتب بودن این ساختار امکان جست و جوی دودویی برای بررسی وجود یال را فراهم می کند، در حالی که ذخیره سازی پیوسته ی داده ها در حافظه باعث افزایش کارایی پیمایش و استفاده ی بهتر از cache پردازنده می شود. این روش به دلیل سادگی، مصرف حافظه ی پایین و کارایی مناسب، برای بخش عمده ای از نودهای گراف گزینه ای مناسب محسوب می شود.

با این حال، برای نودهای سنگین که دارای تعداد بسیار زیادی همسایه هستند، استفاده از یک vector مرتب ساده ممکن است در عملیات به روزرسانی هزینه بر باشد؛ زیرا درج یا حذف عناصر می تواند نیازمند جابه جایی حجم زیادی از داده ها باشد. برای این دسته از نودها می توان از ساختارهای پیشرفته تری مانند chunked sorted vector + delta buffer یا base sorted array + mutable small delta استفاده کرد.

در روش **chunked sorted vector**، لیست همسایگی یک نود سنگین به چند بخش مرتب کوچکتر تقسیم می‌شود. هر بخش یا **chunk** شامل تعدادی از همسایه‌هاست و به‌صورت مرتب نگهداری می‌شود. این طراحی باعث می‌شود عملیات درج یا حذف تنها روی **chunk** مربوطه انجام شود و نیازی به جابه‌جایی کل لیست بزرگ همسایگی نباشد. در نتیجه، این روش برای نودهای با درجه‌ی بسیار بالا مناسب بوده و بین سرعت پیمایش، مصرف حافظه و هزینه‌ی به‌روزرسانی تعادل ایجاد می‌کند.

روش **base sorted array + mutable small delta** نسخه‌ای ساخت‌یافته‌تر از رویکرد مبتنی بر **delta** است. در این روش، بخش اصلی داده‌ها در قالب یک آرایه‌ی مرتب و نسبتاً پایدار ذخیره می‌شود و تغییرات کوچک، شامل درج‌ها و حذف‌ها، در یک **delta** قابل‌تغییر نگهداری می‌شوند. در زمان پاسخ‌گویی به درخواست‌ها، نتیجه‌ی نهایی از ترکیب منطقی **base** و **delta** به دست می‌آید. در صورت افزایش اندازه‌ی **delta**، عملیات **merge** یا **compaction** انجام شده و تغییرات در **base** اعمال می‌شوند. این روش برای نودهای سنگین در گراف‌های نیمه‌پویا مناسب است، زیرا هم سرعت خواندن از ساختار مرتب اصلی را حفظ می‌کند و هم هزینه‌ی به‌روزرسانی‌های کوچک را کاهش می‌دهد.

در روش **vector + delta buffer**، داده‌ی اصلی همسایگی در یک **vector** مرتب نگهداری می‌شود و تغییرات جدید به‌صورت موقت در یک **buffer** جداگانه ثبت می‌گردند. پس از آنکه اندازه‌ی **buffer** از یک آستانه‌ی مشخص عبور کرد، محتوای آن با **vector** اصلی ادغام شده و یک لیست مرتب جدید تولید می‌شود. این روش هزینه‌ی درج‌های مکرر را کاهش می‌دهد و برای گراف‌هایی مناسب است که بیشتر عملیات آن‌ها خواندنی بوده، اما به‌روزرسانی‌های دوره‌ای نیز دارند.

در مجموع، ترکیب **Dense-ID** برای شناسه‌گذاری فشرده‌ی نودها، **sorted vector** برای نودهای معمولی و ساختارهای **chunked** یا **delta-based** برای نودهای سنگین، یک رویکرد کارآمد برای ذخیره‌سازی گراف در **RAM** فراهم می‌کند. این طراحی ضمن کاهش مصرف حافظه، باعث بهبود سرعت پیمایش، افزایش کارایی جست‌وجو و مدیریت بهتر نودهای با درجه‌ی بالا می‌شود.

پیشنهاد نهایی بنده برای ساختمان داده گرافی داخل حافظه RAM با توجه به پیچیدگی حافظه و پیچیدگی زمانی و همچنین چالش های پیاده سازی الگوریتم ساختمان داده زیر است :

Dense-ID + sorted vector + Adjacency List

(۴) مشکل شماره ۴ : مشکلی که در دیتابیس in-memory وجود دارد این است که اطلاعات با بستن برنامه یا خاموشی سیستم از بین می رود این مشکل در یه دیتابیس گرافی شبکه اجتماعی مشکل ساز خواهد بود برای حل این مشکل راه حل پیشنهادی بنده طراحی سیستم ذخیره سازی در دیسک بدون استفاده از دیتابیس آماده است. این سیستم باید به نحوی بهینه باشد تا فرایند درج و حذف با پیچیدگی زمانی پایین رخ دهد از طرفی انتقال اطلاعات از ان به داخل RAM سریع باشد این سیستم همچنین باید از گراف نا همگن پشتیبانی کند.

راه حل پیشنهادی من طراحی سیستمی در لایه دیتابیس هست که دریافت دستور ساخت گراف توسط کاربر که حاوی ویژگی های مانند نوع گراف (پویا / ثابت) ، دوره زمانی اسنپ شات ، ویژگی یکتایی که باید گره شود (برای مثال ID یکتا کاربران) ، تعریف یال (کاربر باید تعیین کند این گره ها در دیتابیس RocksDB چطور با هم ارتباط دارن تا یال ها در گراف تشکیل شود) ، تایپ هر گره برای گراف ناهمگن (گره از نوع کاربر هست یا پست) و

پس از دریافت کوئری کاربر Parser با خواندن ان توابع و متد های لازم را فراخوانی کرده و گراف از داده های داخل RocksDB در ان لحظه بیلد میشود ، پس از بیلد در مکانی که exe. قرار دارد پوشه ایی با اسم گراف ساخته میشود که حاوی فایل های nex. ، .nexr ، graph_meta.nexl. و graph.wal خواهد بود.

```
graph_name/
├─ meta.nexl
├─ graph.wal
├─ nodes_000001.nex
├─ nodes_000002.nex
├─ edges_000001.nexr
├─ edges_000002.nexr
└─ ...|
```

گراف داخل RAM از فایل های nex و nexr که باینری هستن رندر میشود. فایل nex. فایلی برای نگهداری گره هاست که فرمت زیر را دارد :

```
struct NodeRecord {  
    uint64_t node_id;  
    uint32_t type; uint32_t flags;  
    uint64_t out_degree;  
    uint64_t in_degree;  
};
```

در این ساختار node_id شناسه داخلی و متراکم گره در سیستم گراف است. این شناسه از نوع Dense-ID است و برای دسترسی سریع به رکورد گره استفاده می شود. ویژگی type نوع گره را مشخص می کند. برای مثال می تواند بیانگر USER، PAGE، CHANNEL و ... باشد. ویژگی Flags وضعیت گره را نگهداری می کند؛ مانند ACTIVE، DELETED یا سایر فلگ های سیستمی. همچنین ویژگی out_degree تعداد یال های خروجی از این گره را نشان می دهد. در گراف FOLLOW، این مقدار معادل تعداد FOLLOWING است. و همچنین ویژگی in_degree تعداد یال های ورودی به این گره را نشان می دهد. در گراف FOLLOW، این مقدار معادل تعداد FOLLOWERS است. همچنین فایل nexr برای ذخیره سازی یال های گراف است و دارای فرمت ذخیره سازی زیر است

```
struct EdgeRecord {  
    uint64_t edge_id;  
    uint64_t src;  
    uint64_t dst;  
    uint32_t type;  
    uint32_t flags;  
};
```

در این ساختار که برای ذخیره سازی روابط بین گره ها یا به اصطلاح ریاضی یال هست ، دارای ویژگی هایی هست اولین ویژگی edge_id برای شناسه داخلی یال است. دومین ویژگی src برای شناسه گره مبدأ. سومین ویژگی dst هست برای شناسه گره مقصد. ویژگی چهارم type برای نوع رابطه را

مشخص می‌کند. برای مثال در گراف کاربران می‌تواند FOLLOW باشد. ویژگی آخر flags که وضعیت یال را نگهداری می‌کند؛ مانند ACTIVE یا DELETED.

هر دو فایل شامل رکوردهایی با اندازه ثابت هستند. این موضوع چند مزیت مهم دارد:

- امکان دسترسی مستقیم به رکورد با استفاده از offset
- سادگی در خواندن و نوشتن
- سرعت مناسب برای append و overwrite
- سادگی در بازسازی وضعیت حافظه هنگام راه‌اندازی مجدد سیستم

مدل شناسه‌ها: Dense-ID

در این طراحی از Dense-ID استفاده می‌شود.

Dense-ID به این معناست که شناسه‌های داخلی گره‌ها و یال‌ها به صورت متراکم و ترتیبی تخصیص داده می‌شوند.

مزایای این روش:

- دسترسی سریع‌تر به رکوردها
- حذف نیاز به نگاشت‌های پیچیده برای یافتن offset
- مناسب برای بارگذاری مستقیم در RAM
- سادگی در ساخت adjacency structure در حافظه

مدیریت حذف و استفاده مجدد از فضا

از آنجا که سیستم باید روی SSD عملیات درج و حذف را سریع انجام دهد، حذف فیزیکی رکوردها در لحظه انجام نمی‌شود.

در عوض، رکورد حذف‌شده فقط با تغییر flags به وضعیت DELETED علامت‌گذاری می‌شود.

برای استفاده مجدد از این فضاها، دو ساختار stack در حافظه نگهداری می‌شود:

- Node Free Stack
- Edge Free Stack

این **stack** ها شناسه یا موقعیت رکوردهای حذف شده را نگهداری می کنند تا در درج های بعدی دوباره استفاده شوند.

مزایای این روش

- حذف بسیار سریع است
- درج می تواند از فضاهای آزاد قبلی استفاده کند
- نیاز به جابجایی سنگین رکوردها در فایل وجود ندارد
- ساختار روی SSD ساده و کم هزینه باقی می ماند

رفتار Stack های فضای خالی

Stack های مربوط به خانه های خالی **Persist نمی شوند** و روی دیسک ذخیره نمی گردند. این ساختارها فقط در زمان اجرای برنامه در RAM وجود دارند.

در زمان راه اندازی مجدد سیستم:

پس از باز شدن برنامه، این **stack** ها از طریق اسکن مجدد فایل های **graph.nex** و **graph.nexr** ساخته می شوند.

به این صورت:

- اگر رکوردی در وضعیت **DELETED** باشد، شناسه آن وارد **stack** مربوطه می شود.
- اگر رکوردی **ACTIVE** باشد، در ساخت گراف حافظه ای استفاده می شود.
- این تصمیم باعث می شود:
- فرمت فایل ساده باقی بماند
- نیاز به مدیریت فایل کمکی برای **freelist** از بین برود
- ریسک ناسازگاری بین فایل اصلی و **freelist** ذخیره شده حذف شود
- راه اندازی مجدد سیستم **deterministic** و قابل بازسازی باشد

بارگذاری همزمان گراف در RAM (فقط گراف live)

در این معماری، نسخه اصلی پایدار داده روی دیسک قرار دارد، اما برای اجرای **Query** ها، گراف به صورت همزمان در **RAM** نیز ساخته و نگهداری می شود.

هدف از این لایه RAM

- اجرای سریع Queryها
- پیمایش سریع adjacencyها
- پاسخ‌دهی کم‌تاخیر برای عملیات گرافی
- عدم نیاز به query مستقیم از روی فایل‌های SSD

در زمان startup:

هنگام بالا آمدن سیستم:

۱. فایل graph.nex خوانده می‌شود
۲. فایل graph.nexr خوانده می‌شود
۳. رکوردهای ACTIVE در ساختارهای حافظه‌ای بارگذاری می‌شوند
۴. رکوردهای DELETED در stackهای خانه خالی قرار می‌گیرند

در نتیجه، پس از اتمام startup:

- گراف عملیاتی به‌طور کامل در RAM در دسترس است
- فضاهای آزاد قبلی نیز برای درج‌های بعدی قابل استفاده هستند

منطق درج

درج گره

برای ایجاد گره جدید:

۱. ابتدا بررسی می‌شود که آیا Node Free Stack دارای خانه آزاد هست یا نه
۲. اگر خانه آزاد موجود باشد، همان موقعیت مجدداً استفاده می‌شود
۳. در غیر این صورت، رکورد جدید به انتهای فایل graph.nex اضافه می‌شود
۴. همزمان گره در ساختار RAM نیز ایجاد می‌شود

درج یال

برای ایجاد یال جدید:

۱. ابتدا Edge Free Stack بررسی می شود
۲. اگر رکورد حذف شده ای برای استفاده مجدد وجود داشته باشد، همان محل بازنویسی می شود
۳. در غیر این صورت، یال جدید به انتهای فایل graph.nexr اضافه می شود
۴. سپس:

- out_degree گره مبدأ افزایش می یابد
- in_degree گره مقصد افزایش می یابد
- adjacency های RAM به روزرسانی می شوند

منطق حذف

حذف گره

در حذف گره، رکورد مربوطه به صورت منطقی حذف می شود:

- flags به وضعیت DELETED تغییر می کند
 - شناسه آن در Node Free Stack قرار می گیرد
- در صورت نیاز، حذف یال های متصل به این گره می تواند به صورت جداگانه یا در فرآیند cleanup انجام شود.

حذف یال

برای حذف یال:

- رکورد یال به صورت منطقی DELETED می شود
 - edge_id آن در Edge Free Stack قرار می گیرد
 - out_degree و in_degree گره های مربوطه اصلاح می شود
 - گراف RAM نیز همزمان به روزرسانی می شود
- این روش نسبت به حذف فیزیکی مستقیم، بسیار سریع تر و کم هزینه تر است.

مزایای اصلی این طراحی

این طراحی با تمرکز بر سادگی، کارایی و قابلیت بازیابی طراحی شده است. یکی از مهم‌ترین مزیت‌های آن، **سادگی ساختار ذخیره‌سازی** است. در این معماری تنها دو نوع رکورد اصلی وجود دارد: **NodeRecord** برای گره‌ها و **EdgeRecord** برای یال‌ها. هر دو ساختار اندازه ثابت دارند و در فایل‌های اختصاصی خود ذخیره می‌شوند. این موضوع باعث می‌شود فرمت فایل‌ها ساده، قابل فهم، قابل نگهداری و در عین حال مناسب برای پیاده‌سازی‌های با کارایی بالا باشد. سادگی این لایه همچنین ریسک بروز ناسازگاری‌های ساختاری را کاهش می‌دهد و توسعه و عیب‌یابی سیستم را آسان‌تر می‌کند. مزیت مهم دیگر این معماری، **کارایی مناسب برای SSD** است. درج رکوردها عمدتاً به‌صورت **append** یا بازاستفاده از خانه‌های آزاد انجام می‌شود و حذف‌ها نیز به‌صورت منطقی و با استفاده از **flags** مدیریت می‌شوند. در نتیجه، سیستم نیازمند بازنویسی گسترده فایل‌ها یا جابجایی مداوم داده‌ها نیست. این الگو با ویژگی‌های **SSD** سازگار است و موجب می‌شود عملیات درج و حذف با هزینه پایین و سرعت مناسب انجام شوند.

از منظر پایداری و نگهداری، این طراحی دارای **قابلیت بازسازی کامل وضعیت سیستم** است. در هنگام راه‌اندازی مجدد، با اسکن فایل‌های اصلی، هم‌گراف موجود در **RAM** و هم **stack**‌های مربوط به خانه‌های آزاد قابل بازسازی هستند. به این معنا که سیستم برای نگهداری وضعیت عملیاتی خود وابسته به ساختارهای حافظه‌ای **persist** شده نیست و تمام **state** اجرایی به‌صورت **deterministic** از روی داده‌های ذخیره‌شده قابل احیا است. این ویژگی، طراحی را ساده‌تر و بازیابی سیستم را قابل اعتمادتر می‌کند.

این معماری همچنین برای **گراف‌های نامتوازن و پرتراکم** بسیار مناسب است؛ به‌ویژه در سناریوهایی مانند شبکه‌های اجتماعی که در آن برخی گره‌ها تنها تعداد کمی ارتباط دارند، در حالی که برخی دیگر ممکن است میلیون‌ها یال ورودی یا خروجی داشته باشند. از آنجا که یال‌ها به‌صورت رکوردهای مستقل ذخیره می‌شوند و ساختارهای **adjacency** در حافظه ساخته می‌شوند، وجود **super-node**ها یا توزیع نامتوازن درجه‌ها مانع از عملکرد مناسب سیستم نخواهد شد. این موضوع باعث می‌شود طراحی حاضر برای گراف‌های واقعی در مقیاس بالا، عملی و قابل اتکا باشد.

در نهایت، یکی از ویژگی‌های مهم این معماری، جداسازی کامل **topology** گراف از **metadata** است. این لایه صرفاً مسئول ذخیره‌سازی ساختار گراف، یعنی گره‌ها، یال‌ها و **degree**ها است. در مقابل، اطلاعات تجاری، ویژگی‌ها و **metadata** اشیاء در لایه‌ای مستقل مانند **RocksDB** نگهداری می‌شوند. این جداسازی مزایای متعددی دارد: لایه گراف سبک و تخصصی باقی می‌ماند، تغییرات در **metadata** به طراحی فایل‌های گراف وابسته نمی‌شود، و توسعه یا بهینه‌سازی هر لایه می‌تواند به‌صورت مستقل انجام گیرد. در نتیجه، معماری کلی سیستم شفاف‌تر، ماژولارتر و قابل توسعه‌تر خواهد بود.

فایل **graph_meta.nexl** (معماری این فایل هنوز کامل نیست)

فایل **graph_meta.nexl** مسئول نگهداری متادیتای سیستمی گراف است و بخشی از **state** کنترلی **engine** را ثبت می‌کند. این فایل شامل داده‌های کاربردی گراف نیست، بلکه اطلاعاتی را نگهداری می‌کند که برای **setup**، **validation** و **recovery** سیستم ضروری هستند. به‌صورت مفهومی، این فایل می‌تواند شامل اطلاعاتی از جنس زیر باشد:

- نسخه فرمت فایل‌های گراف
- شناسه یا نام گراف
- زمان آخرین **rebuild** کامل
- زمان آخرین **startup** موفق
- تعداد تقریبی **node** و **edge**
- وضعیت **consistency** بین **RocksDB** و فایل‌های گراف
- فلگ یا **marker** مربوط به نیاز به **rebuild**
- وضعیت **shutdown** قبلی (**clean / unclean**)

مهم‌ترین نقش **graph_meta.nexl** در این طراحی، فراهم کردن مکانیزمی برای تشخیص نیاز به **rebuild** فایل‌های **graph.nex** و **graph.nexr** از روی داده‌های موجود در **RocksDB** در زمان **setup** است.

به این صورت که هنگام راه اندازی سیستم، engine ابتدا metadata را از graph_meta.nextl می خواند و بررسی می کند که آیا فایل های فیزیکی گراف معتبر و قابل استفاده هستند یا خیر. اگر بر اساس فلگ های ثبت شده، شماره نسخه، وضعیت shutdown، marker های consistency یا هر سیاست کنترلی دیگر مشخص شود که فایل های graph.nex و graph.nexr ممکن است قدیمی، ناقص یا ناسازگار باشند، سیستم وارد فرآیند rebuild می شود. در این فرآیند، ساختار گراف مجدداً از روی داده های موجود در RocksDB ساخته می شود و فایل های باینری گراف از ابتدا تولید می گردند. این قابلیت باعث می شود سیستم بتواند در شرایطی مانند موارد زیر به صورت کنترل شده عمل کند:

- خرابی یا ناقص بودن shutdown قبلی
 - ناسازگاری بین state ثبت شده در RocksDB و فایل های گراف
 - تغییر نسخه ساختار داخلی فایل ها
 - نیاز به بازتولید کامل گراف در deployment یا migration
- در نتیجه، graph_meta.nextl نقش مهمی در اعتبارسنجی startup و تصمیم گیری برای استفاده از فایل های فعلی یا rebuild کامل ایفا می کند.

فایل graph.wal (ساختار این فایل هنوز کامل نیست)

فایل graph.wal به عنوان **Write-Ahead Log** برای ثبت عملیات سیستمی و پشتیبانی از recovery در نظر گرفته می شود. هدف از این فایل آن است که تغییرات مهم پیش از تثبیت کامل در لایه ذخیره سازی نهایی، در قالب رکوردهای لاگ ثبت شوند تا در صورت بروز crash یا shutdown غیرمنتظره، engine بتواند وضعیت صحیح را بازسازی کند.

در این معماری، از آنجا که topology گراف در فایل های graph.nex و graph.nexr نگهداری می شود اما داده مرجع و metadata در RocksDB قرار دارد، ترتیب اعمال عملیات باید مشخص و کنترل شده باشد.

ترتیب اعمال عملیات در سیستم

در عملیات هایی که منجر به ایجاد یا تغییر در داده های گراف می شوند، ترتیب کلی اعمال به صورت زیر در نظر گرفته می شود:

۱. ابتدا داده یا تغییر مرجع در RocksDB ثبت می‌شود
 ۲. سپس ساختار RAM به‌روزرسانی می‌شود
 ۳. در نهایت تغییر روی فایل‌های SSD (graph.nexr / graph.nex) اعمال می‌شود
- این ترتیب از این اصل تبعیت می‌کند که RocksDB به‌عنوان مرجع اصلی داده و source of truth در نظر گرفته می‌شود و لایه فایل‌های گراف، representation تخصصی و بهینه‌شده‌ای از topology برای پردازش سریع است.
- بنابراین، اگر در میانه عملیات مشکلی رخ دهد، سیستم می‌تواند با اتکا به RocksDB و اطلاعات ثبت‌شده در WAL، وضعیت را اصلاح یا بازسازی کند.

نقش WAL در این ترتیب

graph.wal رکوردهای لازم برای عملیات‌هایی مانند موارد زیر را ثبت می‌کند:

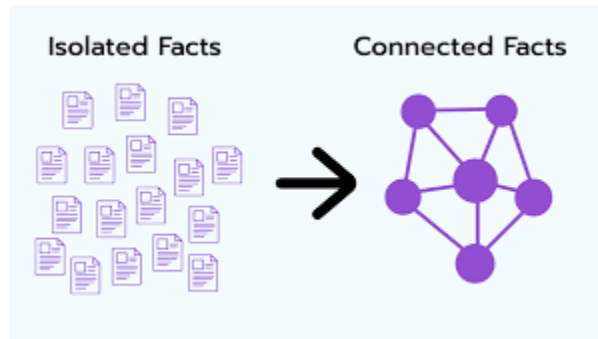
- ایجاد گره
 - حذف گره
 - ایجاد یال
 - حذف یال
 - تغییر وضعیت رکوردها
 - شروع یا پایان عملیات rebuild
 - markerهای مربوط به commit یا completion عملیات
- در عمل، WAL تضمین می‌کند که اگر سیستم پس از ثبت تغییر در RocksDB و پیش از تکمیل به‌روزرسانی RAM یا SSD متوقف شود، در startup بعدی بتوان تشخیص داد چه عملیاتی به‌طور کامل نهایی نشده‌اند و باید replay، تکمیل یا rebuild شوند.

سناریوی منطقی استفاده از WAL

برای مثال، در عملیات ایجاد یال:

- ابتدا موجودیت یا ارتباط مرجع در RocksDB ذخیره می‌شود
- سپس عملیات در graph.wal ثبت می‌شود
- بعد گراف در RAM به‌روزرسانی می‌شود
- و در نهایت رکورد EdgeRecord در فایل graph.nexr نوشته می‌شود

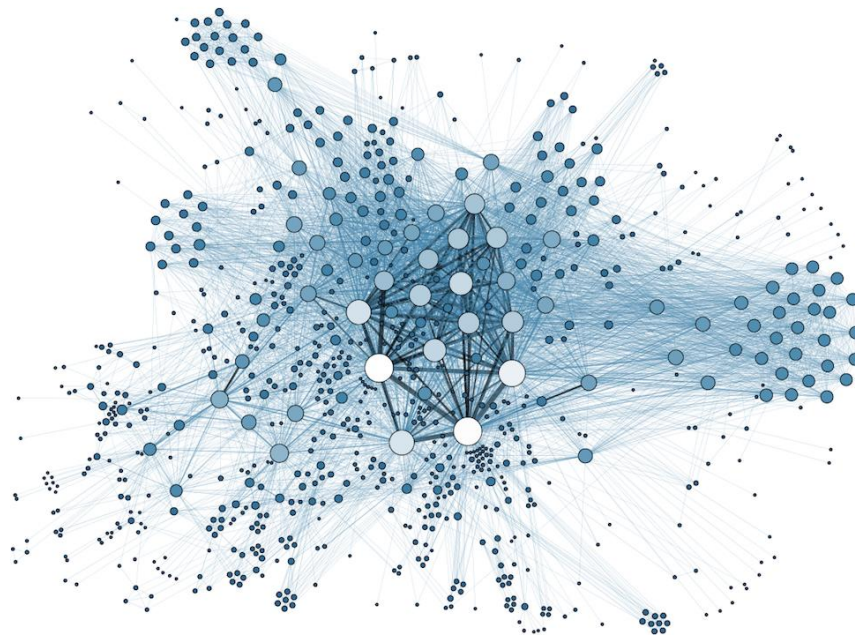
اگر سیستم در هر نقطه قبل از تکمیل نهایی متوقف شود، engine در startup بعدی می‌تواند با خواندن WAL و بررسی metadata، تشخیص دهد که آیا عملیات باید تکمیل شود، rollback منطقی گردد، یا rebuild انجام شود.



۵) مشکل شماره ۵: در شبکه‌های اجتماعی بزرگ، ساختار داده معمولاً به شکل یک گراف عظیم و پویا مدل می‌شود. در این گراف، کاربران، صفحات، گروه‌ها و محتواها به عنوان گره و روابطی مانند دوستی، دنبال کردن، لایک، اشتراک‌گذاری و کامنت به عنوان یال نمایش داده می‌شوند.

ویژگی کلیدی این گراف، پویایی مداوم آن است؛ به این معنا که در هر لحظه، تعداد زیادی رابطه ایجاد یا حذف می‌شود و ویژگی‌های گره‌ها و یال‌ها تغییر می‌کنند. این تغییرات دائمی باعث می‌شود اجرای کوئری روی چنین گرافی از نظر سازگاری، کارایی و مقیاس‌پذیری به مسئله‌ای پیچیده تبدیل شود.

در این محیط، همه کوئری‌ها رفتار یکسانی ندارند. برخی کوئری‌ها بسیار ساده و کوتاه هستند و باید با حداقل تأخیر پاسخ داده شوند، در حالی که برخی دیگر محاسباتی، سنگین و زمان‌بر هستند و اجرای آن‌ها روی داده زنده می‌تواند باعث کاهش شدید کارایی سیستم شود. به همین دلیل، استفاده از یک رویکرد یکسان برای همه انواع کوئری معمولاً بهینه نیست و لازم است workload سیستم بر اساس نوع کوئری تفکیک شود.



چالش‌های اصلی کوئری‌زدن روی گراف داینامیک عظیم

۱. تغییر مداوم داده در حین اجرای کوئری

در زمان اجرای کوئری، گراف به‌طور پیوسته در حال تغییر است. اگر کوئری در چند مرحله داده بخواند، ممکن است هر مرحله را از نسخه متفاوتی از گراف دریافت کند. این موضوع باعث می‌شود نتیجه نهایی ناسازگار یا غیرقابل اتکا باشد.

۲. تداخل بین خواندن و نوشتن

در شبکه‌های اجتماعی، عملیات خواندن و نوشتن هر دو با نرخ بالا اتفاق می‌افتند. اگر سیستم کنترل مناسبی روی این تداخل نداشته باشد، یا خواندن‌ها داده ناپایدار مشاهده می‌کنند، یا نوشتن‌ها با تأخیر زیاد مواجه می‌شوند.

۳. افزایش latency در کوئری‌های سنگین

اگر کوئری‌های سنگین مستقیماً روی داده زنده اجرا شوند، نه تنها خودشان زمان زیادی مصرف می‌کنند، بلکه ممکن است منابع لازم برای پاسخ‌دهی سریع به کوئری‌های ساده‌تر را نیز اشغال کنند. این موضوع کیفیت سرویس را کاهش می‌دهد.

راهکار اول: Snapshot

تعریف

Snapshot به معنای اجرای کوثری روی یک نمای ثابت و سازگار از گراف در یک لحظه مشخص است. در این روش، وقتی کوثری آغاز می‌شود، یک نسخه منطقی از گراف برای آن در نظر گرفته می‌شود و تمام مراحل کوثری فقط همان نسخه را مشاهده می‌کنند.

مزایا

- ارائه نتیجه سازگار و قابل تکرار
- مناسب برای کوثری‌های طولانی و چندمرحله‌ای
- جلوگیری از تأثیر تغییرات لحظه‌ای روی نتیجه نهایی
- مناسب برای پردازش‌های تحلیلی و سنگین

معایب

- نیاز به نگهداری نسخه‌های مختلف از داده
- افزایش هزینه ذخیره‌سازی و مدیریت نسخه
- احتمال استفاده از داده‌ای که کاملاً به‌روز نیست

راهکار دوم: Lock

تعریف

Lock روشی برای کنترل هم‌زمانی است که در آن بخشی از داده هنگام اجرای عملیات قفل می‌شود تا از تداخل خواندن و نوشتن جلوگیری شود. این قفل می‌تواند در سطح گره، یال، پارتیشن یا زیرگراف اعمال شود.

مزایا

- سازگاری قوی در سطح داده زنده
- مناسب برای کوثری‌ها و تراکنش‌های کوتاه
- مدل قابل فهم و متداول در سیستم‌های تراکنشی

معایب

- کاهش هم‌زمانی در بارهای سنگین
- ایجاد انتظار برای عملیات دیگر
- احتمال deadlock
- مناسب نبودن برای کوئری‌های طولانی

نقش Job در اجرای کوئری‌های سنگین

تعریف Job

در این زمینه، **Job** به یک پردازش غیرهم‌زمان یا پس‌زمینه‌ای گفته می‌شود که برای اجرای کوئری‌ها یا محاسبات سنگین استفاده می‌شود.

به‌جای آن‌که یک کوئری پیچیده مستقیماً در مسیر پاسخ‌گویی بلادرنگ اجرا شود، سیستم آن را به‌عنوان یک job ثبت می‌کند تا در پس‌زمینه، با منابع کنترل‌شده و روی یک snapshot مشخص اجرا شود.

به بیان دیگر، job یک سازوکار اجرایی برای جدا کردن پردازش‌های سنگین از مسیر درخواست‌های آنلاین است.

چرایی استفاده از Job

اگر همه کوئری‌ها، چه ساده و چه پیچیده، مستقیماً روی گراف زنده اجرا شوند، سیستم با مشکلات زیر مواجه می‌شود:

- افزایش شدید latency
 - اشغال منابع محاسباتی
 - کاهش throughput
 - تداخل بین نیازهای real-time و تحلیل‌های زمان‌بر
- استفاده از job باعث می‌شود کوئری‌های سنگین از مسیر سرویس‌دهی لحظه‌ای جدا شوند.

نحوه عملکرد Job در این معماری

در رویکرد پیشنهادی، زمانی که یک درخواست سنگین وارد سیستم می‌شود، به‌جای اجرای مستقیم، مراحل زیر انجام می‌شود:

۱. درخواست به‌عنوان یک job ثبت می‌شود.
۲. برای job یک snapshot مشخص از گراف انتخاب می‌شود.
۳. job در پس‌زمینه و خارج از مسیر درخواست‌های بلادرنگ اجرا می‌شود.
۴. نتیجه در یک محل میانی مانند table، cache، یا storage تحلیلی ذخیره می‌شود.
۵. کاربر یا سرویس دیگر بعداً نتیجه را بازیابی می‌کند.

برای مثال، کوئری «یافتن کاربران تأثیرگذار در یک اجتماع» ممکن است محاسبات زیادی نیاز داشته باشد.

اجرای این نوع کوئری در لحظه و روی داده زنده، هم پرهزینه است و هم ممکن است باعث افت پاسخ‌گویی سیستم شود.

در عوض، می‌توان آن را به‌عنوان یک job پس‌زمینه‌ای روی snapshot اجرا کرد و نتیجه را بعداً در اختیار کاربر قرار داد.

مزایای استفاده از Job

- جداسازی workload تحلیلی از workload عملیاتی
- کنترل بهتر منابع CPU، memory و I/O
- جلوگیری از فشار مستقیم روی سیستم آنلاین
- مناسب برای کوئری‌های سنگین، تکرارشونده یا batch-like
- امکان زمان‌بندی، اولویت‌بندی و retry

محدودیت‌های Job

- نتیجه معمولاً فوری نیست
- ممکن است خروجی با کمی تأخیر در دسترس قرار گیرد

رویکرد ترکیبی: Snapshot برای Jobهای سنگین و Lock برای

کوئری‌های ساده

ایده اصلی

از آنجا که کوئری‌ها در شبکه اجتماعی از نظر پیچیدگی و نیازمندی متفاوت‌اند، یک رویکرد مناسب می‌تواند ترکیب Snapshot، Lock و Job باشد.

در این مدل:

- کوئری‌های ساده، کوتاه و حساس به زمان با استفاده از lock و مستقیماً روی داده زنده اجرا می‌شوند.
- کوئری‌های سنگین، تحلیلی و زمان‌بر به صورت job در پس‌زمینه و روی snapshot اجرا می‌شوند.

دسته‌بندی کوئری‌ها

۱. کوئری‌های ساده و کوتاه

این گروه شامل کوئری‌هایی است که:

- محدوده کوچکی از گراف را می‌خوانند
 - زمان اجرای کوتاهی دارند
 - باید سریع پاسخ داده شوند
 - معمولاً برای تعاملات روزمره کاربر استفاده می‌شوند
- نمونه‌ها:

- شمارش دوستان یا دنبال‌کنندگان
- نمایش پروفایل و ارتباطات نزدیک
- بررسی وجود رابطه بین دو کاربر
- خواندن لیست کوتاه همسایه‌ها

برای این کوئری‌ها، lock در سطح محدود می‌تواند سازگاری لازم را با هزینه قابل قبول فراهم کند.

۲. کوئری‌های سنگین و بلندمدت

این گروه شامل کوئری‌هایی است که:

- نیاز به traversal چندمرحله‌ای دارند
- حجم بزرگی از گراف را می‌خوانند
- زمان اجرای طولانی دارند
- خروجی آن‌ها معمولاً تحلیلی یا توصیه‌ای است

مزایای رویکرد ترکیبی

۱. بهبود کارایی کلی سیستم

با جدا کردن کوئری‌های سنگین از مسیر آنلاین، منابع سیستم بهتر مدیریت می‌شوند و پاسخ‌گویی به درخواست‌های روزمره سریع‌تر باقی می‌ماند.

۲. تعادل بین consistency و freshness

کوئری‌های ساده که به داده تازه نیاز دارند، روی داده زنده و با lock اجرا می‌شوند. در مقابل، کوئری‌های سنگین که سازگاری کامل برایشان مهم‌تر از تازگی مطلق داده است، روی snapshot اجرا می‌شوند.

۳. مقیاس‌پذیری بهتر

این تفکیک باعث می‌شود سیستم آنلاین و سیستم پردازش پس‌زمینه بتوانند مستقل‌تر مقیاس پیدا کنند و بار هرکدام جداگانه کنترل شود.

۴. کاهش اثر کوئری‌های تحلیلی بر تجربه کاربر

اگر پردازش‌های سنگین در پس‌زمینه اجرا شوند، کاربران نهایی در تعاملات عادی خود کاهش سرعت کمتری احساس خواهند کرد.

۵. مدیریت بهتر اولویت‌ها

در معماری مبتنی بر job، می‌توان کوئری‌های مهم‌تر را در اولویت بالاتر قرار داد، آن‌ها را زمان‌بندی کرد، یا در صورت خطا دوباره اجرا نمود.

محدودیت‌ها و ملاحظات اجرایی

۱. نیاز به مکانیزم تشخیص نوع کوئری

سیستم باید بتواند تشخیص دهد که یک درخواست در دسته کوئری ساده قرار می‌گیرد یا باید به صورت job اجرا شود.

۲. هزینه نگهداری snapshot

برای اجرای jobها روی snapshot، باید زیرساخت نسخه‌بندی مناسبی وجود داشته باشد.

۳. مدیریت صف jobها

در صورت افزایش تعداد jobها، باید مکانیزم صف، زمان‌بندی، مانیتورینگ و کنترل منابع به خوبی طراحی شود.